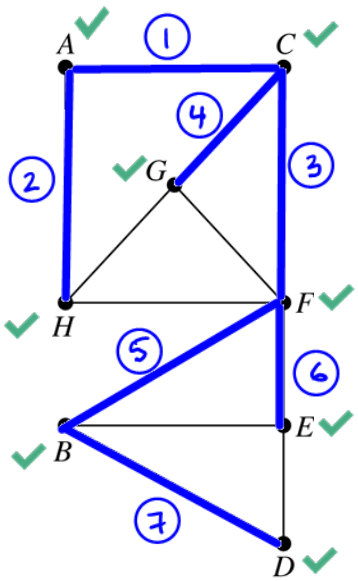
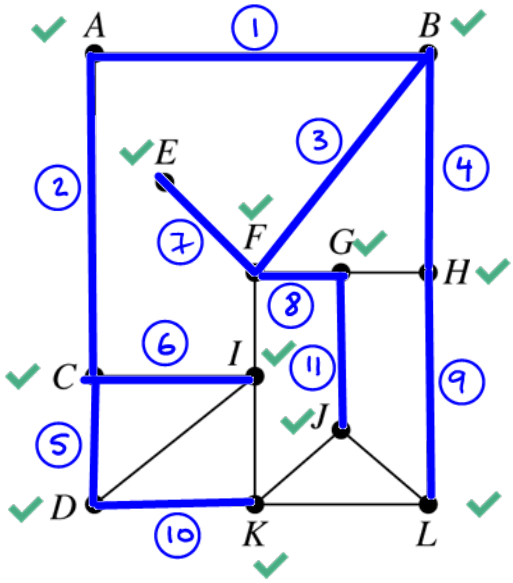
1a. Perform BFS on the following two graphs, starting at vertex A. Show queue Q and give a label to each edge according to the order it is added to the spanning tree (i.e. edge that is added first label 1, edge that is added second label 2, etc.).

Q: A C H F G B E D



Q: A B C F H D I E G L K J



Breadth – First Search Algorithm.

Insert vertex v_x at the rear of (initially empty) queue Q and initialize T as the tree made up of this one vertex v_x. Visit v_x.

```
While (Q is not empty){
        Delete the vertex v from the front of the Q
        for each neighbor w of v  //in the increasing order
                if w is unvisited
                {
                visit(w);
                insert w at the rear of the Q;
                add edge vw to tree T
                }
}
```

	TREE	GRAPH (DISTANCE)
A J	4	4
A G	3	3
I F	4	1

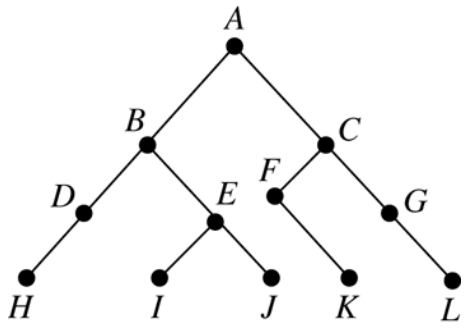
1b. Using the BFS tree, what is the length of the path between vertices A and J? A and G? I and F?

Compare the results with the *distance* between the vertices. What property do you conclude about BFS? BFS finds has shortest paths from the source vertex (A in our case) to all other vertices in the graph

1c. Explain how to utilize BFS algorithm to determine the number of connected components in a graph.

```
num_components = 0
while (there is unvisited vertex Vx) {
    num_components = num_components + 1
    BFS(Vx)
}
print num_components
```

2a. For the tree shown, list the vertices according to a preorder traversal, an inorder traversal, and postorder traversal.

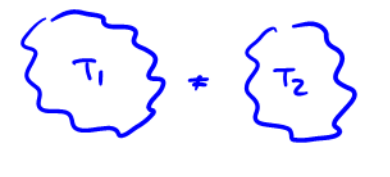


Pre-Order: A B D H E I J C F K G L

In-Order: H D B I E J A F K C G L

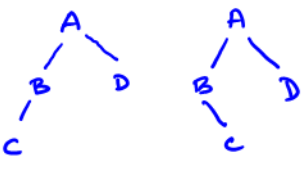
Post-Order: H D I J E B K F L G C A

2b. Construct two distinct binary trees, each having four vertices, so that each has the same preorder listing and the same postorder listing as the other.



PRE (T1) = PRE (T2)

POST (T1) = POST (T2)



PRE : A B C D

POST : C B D A

PRE : A B C D

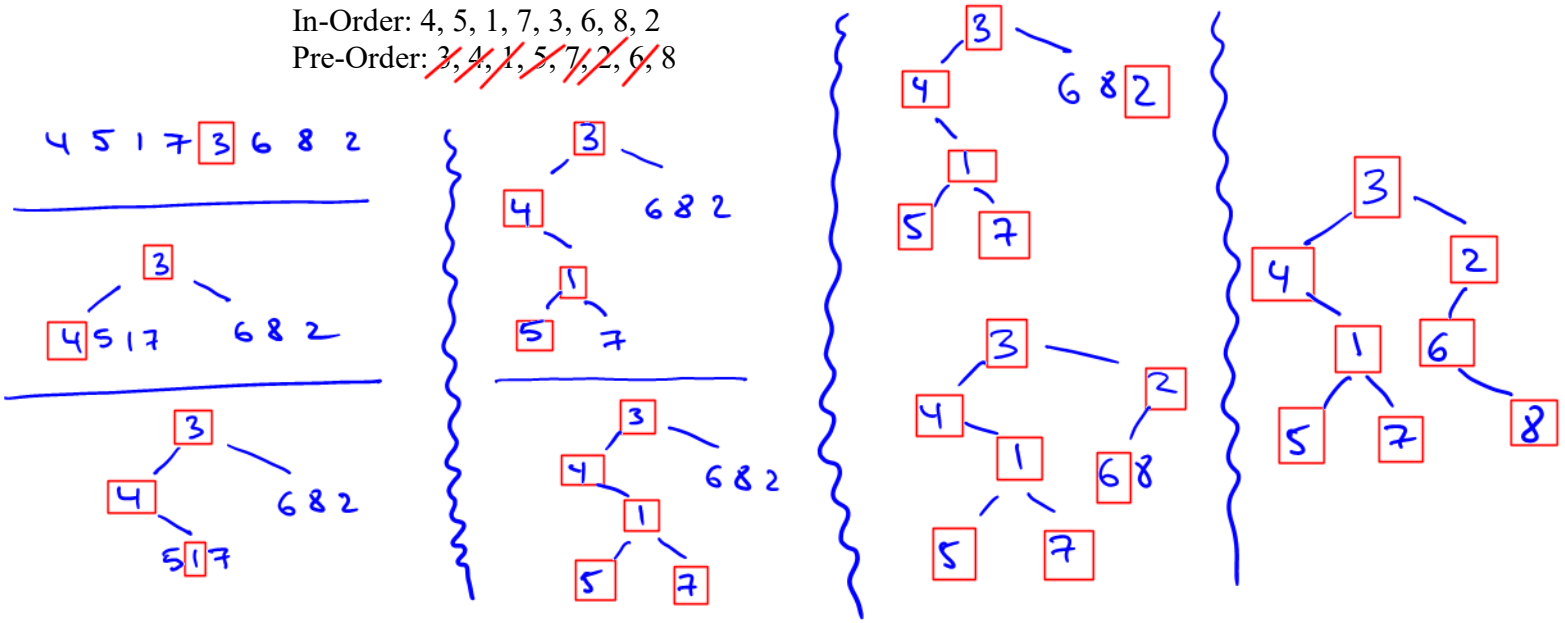
POST : D C B A

in reverse, given PRE & POST order does not imply a UNIQUE TREE

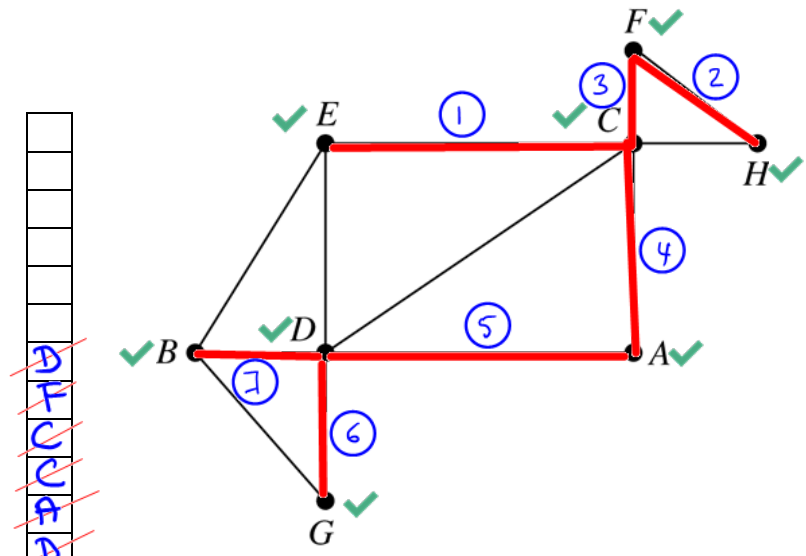
PRE & IN or POST & IN imply a unique TREE

2c. Given the following inorder and preorder of the tree T, recover the tree:

In-Order: 4, 5, 1, 7, 3, 6, 8, 2
Pre-Order: ~~3, 4, 1, 5, 7, 2, 6, 8~~



3a. Do DFS on the following graph, starting at vertex B. Show Return Stack and give a label to each edge according to the order it is added to the spanning tree.



Depth – First Search Algorithm

```
dfs(vertex v)
    {
    visit(v);
    for each neighbor w of v   //in the increasing order
          if w is unvisited
          {
          dfs(w);
          add edge vw to tree T
          }
    }
```

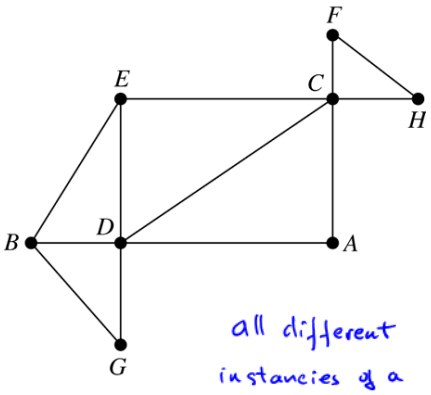
RETURN STACK

← Remember, this is NOT call stack.

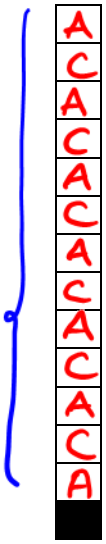
vertices G, E, H are not on the return stack.

3b. Do modified DFS on the following graph, starting at vertex A. Show Return Stack and give a label to each edge according to the order it is added to the spanning tree.

```
dfs(vertex v)
   {
   // visit(v);
   for each neighbor w of v   //in the increasing order
         if w is unvisited
         {
         dfs(w);
         add edge vw to tree T
         }
   }
```



all different instancies of a function

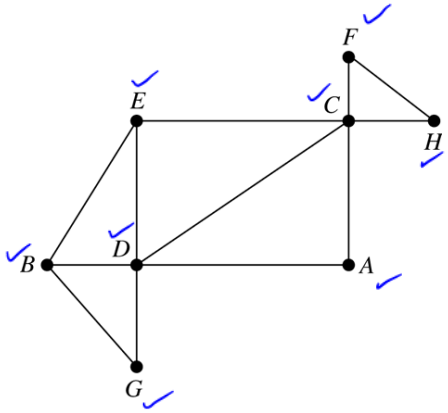


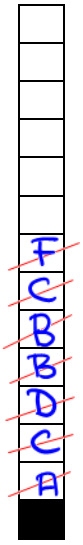
stack overflow

RETURN STACK

3c. Do modified DFS on the following graph, starting at vertex A. Show Return Stack and list of vertices printed on the screen, in the order they are printed.

```
dfs(vertex v)
   {
   visit(v);
   for each neighbor w of v   //in the increasing order
         if w is unvisited
         {
         print w
         dfs(w);
         print v
         }
   }
```





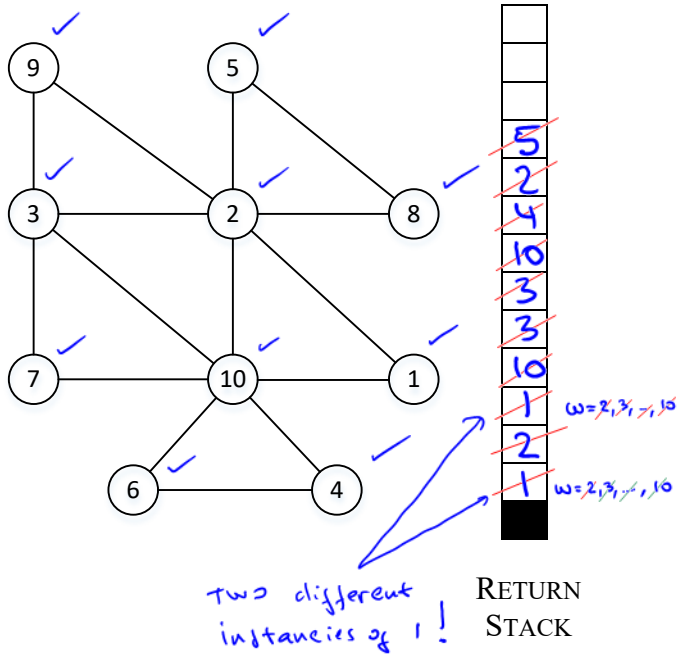
RETURN STACK

PRINTED ON THE SCREEN

C D B E B G B D C F H F C A

3d. Do modified DFS on the following graph, starting at vertex 1, i.e. dfs(1). Show Return Stack and list of vertices printed on the screen, in the order they are printed.

```
dfs(vertex v)
   {
      for each neighbor w of v  //in the increasing order
         if w is unvisited
         {
         visit(w);
         print w
         dfs(w);
         print v
         }
   }
```



PRINTED ON THE SCREEN

RETURN STACK

2 1 10 3 7 3 9 3 10 4 6 4 10 1 2 5 8 5 2 1